

BLOC 3 CDA · DÉPLOYER ET SÉCURISER LES APPLICATIONS INFORMATIQUES

CESIZen

Plan de déploiement, maintenance et sécurisation

Louis Beaujoin — Promotion 2024-2025

Soutenance Bloc 3 · Juillet 2026

github.com/louisbeaujoin/cesizen

Sommaire

1

Contexte et objectifs

Présentation de CESIZen, stack technique, fonctionnalités

2

Plan de déploiement

3 environnements, architecture, Git Flow, CI/CD, Docker

3

Plan de maintenance

Ticketing GitHub, SLA, veille technologique Dependabot

4

Plan de sécurisation

OWASP Top 10, matrice de risques, correctifs, RGPD, crise

5

Démonstration

CI/CD en live, ticketing sur GitHub Projects

6

Conclusion

Bilan, chiffres clés, perspectives

CESIZen — Plateforme de bien-être mental

Le projet

Commande d'un Ministère fictif pour une plateforme web de bien-être mental accessible à tous les citoyens, avec un espace personnel et un back-office administrateur.

Budget : 75 000 € TTC

Durée : 12 mois

Livrable : Application web + API REST Android

Fonctionnalités clés

- Authentification (inscription / connexion / profil)
- Pages d'information santé mentale
- Widget de respiration guidée animé
- Back-office admin (utilisateurs, contenus)
- API REST pour l'application Android

Stack technique

Composant	Technologie
Back-end	Laravel 13 / PHP 8.3
Front-end	Blade + Tailwind CSS 4
Build	Vite 7
Base de données	MySQL 8
Tests	Pest 4 — 44 tests
Conteneur	Docker + PHP 8.3 Apache
CI/CD	GitHub Actions

Les 3 environnements

DEV — Développement

Infrastructure : WAMP local Windows

PHP : 8.3 · **MySQL** : 8.0

Accès : localhost (développeur seul)

Expérimentation, code en cours, tests unitaires locaux

TEST Opérationnel

Infrastructure : Docker Compose local

app : PHP 8.3 Apache — port 8080

db : MySQL 8 — volume persistant

Validation fonctionnelle, démo jury
make up → make seed

PROD — Production (théorique)

Infrastructure : VPS Debian 12

Nginx : HTTPS + PHP-FPM

Domaine : cesizen.fr

Décrit architecturalement — hors périmètre formation
(pas de VPS réel)

Principe : DEV → TEST (Docker, recette) → PROD (merge main + tag de version). Chaque environnement a ses propres variables d'environnement via `.env`, jamais versionné dans git.

Stratégie Git — Git Flow simplifié

Structure des branches

Branche	Rôle
main	Code stable — livrable en prod
develop	Intégration continue des dev
feature/*	Fonctionnalité isolée

Merge via Pull Request uniquement

- Revue de code obligatoire
- Tests CI doivent passer

Conventional Commits

Type	Exemple
feat	feat(auth): connexion OAuth
fix	fix(login): erreur 500 throttle
ci	ci: build Docker ajouté
chore	chore: maj Laravel 13.x

Semantic Versioning

- **v1.0.0** — Livraison initiale
- **v1.1.0** — Nouvelle fonctionnalité
- **v1.0.1** — Correctif de bug

Pipeline CI/CD — GitHub Actions

Déclencheurs

- Push sur main ou develop
- Pull Request vers main

Job 1 — Tests Pest

1. PHP 8.3 + extensions (pdo, sqlite...)
2. Cache Composer (accélération)
3. composer install
4. cp .env.example .env + key:generate
5. composer test
 - 44 tests, SQLite en mémoire, ~2s

Si les 44 tests passent → Job 2 démarre

Job 2 — Build image Docker

```
needs: tests ← bloquant

1. Checkout du code
2. docker build \
  -t cesizen:{SHA_COMMIT} .
   → PHP 8.3 Apache
   → Node 20 + Vite build
   → Composer prod (sans --dev)
```

Garantie : si un test échoue → l'image Docker n'est pas construite. Aucun code cassé n'atteint l'environnement de test.

Durée totale

~2 min 30 — feedback immédiat après chaque push

Configuration Docker — Environnement TEST

Docker Compose — 2 services

```
services:
  app: # PHP 8.3 Apache
    ports: "8080:80"
    depends_on: db (healthcheck)

  db: # MySQL 8.0
    image: mysql:8.0
    volume: db_data (persistant)
```

Commandes Makefile

Commande	Action
make up	Build + démarrage
make seed	Migrations + données démo
make shell	Terminal dans le conteneur
make reset	Redémarrage propre

Dockerfile — étapes

```
FROM php:8.3-apache

# Node.js 20 pour Vite
RUN curl -fsSL .../setup_20.x | bash

# Extensions PHP
RUN docker-php-ext-install \
    pdo pdo_mysql mbstring zip bcmath

# Composer 2
COPY --from=composer:2 /usr/bin/composer .

# Dépendances production
RUN composer install --no-dev
RUN npm ci && npm run build
RUN rm -rf node_modules

# Entrypoint : key:generate + migrate
```

localhost:8080 → application complète en 1 commande

Ticketing — GitHub Issues + Projects

Pourquoi GitHub Issues ?

Tout est centralisé dans **un seul outil** : code + CI/CD + tickets → traçabilité maximale. Chaque ticket peut être lié directement au commit et à la PR qui le corrigent.

Labels configurés

Catégorie	Labels
Type	bug · evolution · maintenance · dependabot
Criticité	bloquant-critique · bloquant-fort · majeur · mineur
Statut	en-cours · en-test

Board GitHub Projects — 4 colonnes

Colonne	Signification
À traiter	Nouveau ticket non assigné
En cours	Développement en cours
En test	Recette en environnement TEST
Terminé	Validé et mergé dans main

Templates : Bug report (pré-rempli avec SLA) et Feature request — réduisent le temps de qualification de 50 %

SLA — Niveaux de service contractuels

Criticité	Définition	Prise en compte	Correction	Exemple
Bloquant critique	Application inaccessible ou données compromises	1 h ouvrée	3 h ouvrées	Serveur hors ligne, fuite de données
Bloquant fort	Fonctionnalité majeure KO, sans contournement	2 h ouvrées	6 h ouvrées	Connexion impossible
Majeur	Gêne importante mais contournement possible	7 h ouvrées	16 h ouvrées	Erreur affichage mobile
Mineur	Inconfort visuel ou ergonomique	1 j. ouvré	40 h ouvrées	Faute d'orthographe

Canal d'urgence : email direct pour tout incident bloquant critique — pas d'attente de la prochaine consultation du board GitHub.

Communication mensuelle : export du board GitHub Projects → compte-rendu au Ministère + réunion de priorisation.

Veille technologique — Dependabot + sources

Dependabot — automatisation des mises à jour

Périmètre	Fréquence	Cible
Composer (PHP)	Hebdomadaire	Lundi 08h00
npm (JS)	Hebdomadaire	Lundi 08h00
GitHub Actions	Mensuelle	1er du mois

Processus : Dependabot ouvre une PR automatique → tests CI s'exécutent → merge si tout est vert

Sources de veille manuelle

Source	Fréquence
Laravel News	Hebdomadaire
CHANGELOG Laravel (GitHub)	À chaque release
Bulletins ANSSI / CVE	Hebdomadaire
PHP RFC	Mensuelle

Bénéfice clé : 0 heure passée à détecter les vulnérabilités connues — Dependabot les remonte automatiquement.

Analyse OWASP Top 10 — Couverture du code réel

Protections natives de Laravel

OWASP	Mécanisme
A03 — Injection SQL	Eloquent ORM → requêtes préparées
A03 — XSS HTML	Blade {{ }} → htmlspecialchars auto
A01 — CSRF	@csrf dans chaque formulaire
A02 — Mots de passe	Cast 'hashed' + bcrypt 12 rounds
A01 — Contrôle d'accès	Middleware auth + admin double
A07 — Session	session()->regenerate() post-login

Corrections ajoutées

Risque	Correctif
Brute force (A07)	throttle:5,1 sur /connexion et /api/login
XSS paramètres JS (A03)	Cast (int) sur durées dans BreathingController
APP_DEBUG=true (A05)	.env.example → APP_DEBUG=false
En-têtes HTTP absents (A05)	SecurityHeadersMiddleware sur toutes les routes web

10 risques sur 12 couverts = 83 %

Objectif contractuel : 75 % 

Matrice de risques et correctifs appliqués

#	Risque	Prob. /5	Impact /5	Score /25	Statut	Action
R1	Brute force login	4	5	20	✓ Corrigé	throttle:5,1
R2	XSS params URL → <script>	3	4	12	✓ Corrigé	Cast (int)
R4	APP_DEBUG=true en prod	3	3	9	✓ Corrigé	.env.example false
R5	Absence en-têtes HTTP	4	2	8	✓ Corrigé	SecurityHeadersMiddleware
R3	Token API stocké en clair	2	4	8	⚠ Recommandé	Hachage Token (prochaine v)
R6	Injection SQL	1	5	5	✓ Natif	Eloquent ORM
R9	Mots de passe faibles	1	5	5	✓ Natif	bcrypt 12 rounds

Résultat : 10 risques sur 12 couverts = **83 %** de couverture (objectif 75 % dépassé). 1 risque mineur (R3) planifié pour v1.1.0.

RGPD et plan de gestion de crise

RGPD — Données collectées

Donnée	Finalité	Durée
Nom + e-mail	Authentification	Compte + 1 an
Sessions respiration	Suivi personnel	2 ans glissants
ID de session	Connexion	120 minutes

Droits mis en œuvre

- **Accès / Rectification** : page profil /profil
- **Effacement** : suppression par l'admin (v1.0)
- **Portabilité** : export CSV planifié (v1.1)

Plan de crise — 7 étapes

T+	Étape
0	Détection — monitoring / signalement
1h	Qualification — gravité + périmètre
2h	Notification — DSI + DPO Ministère
2h	Isolation — mise hors ligne si nécessaire
72h	CNIL — déclaration obligatoire si données exposées
72h	Utilisateurs — email si risque élevé
7j	Post-incident — analyse + remédiation

Démonstration en direct

Partie A — CI/CD (3 min)

1. **Historique Git** : montrer les commits conventionnels sur github.com/louisbeaujoin/cesizen
2. **Pipeline Actions** : ouvrir le dernier workflow → Job 1 Tests (44 passed) → Job 2 Docker (image construite)
3. **Push live** : modifier une ligne → `git commit` → `git push` → pipeline démarre en quelques secondes

"Le pipeline garantit que seul du code testé peut être déployé."

Partie B — Ticketing (3 min)

1. **Board Projects** : montrer les 4 colonnes et tickets positionnés
2. **Ticket #13** : bug bloquant critique — template rempli, labels, SLA affiché
3. **Ticket #16** : Dependabot — colonne Terminé (MAJ Laravel mergée automatiquement)

"L'avantage : code + CI + tickets dans un seul outil → traçabilité totale."

Si le jury souhaite tester Docker : `make up` → `make seed` → `localhost:8080` — application complète en moins de 2 minutes.

Bilan et chiffres clés

44

Tests automatisés Pest
100 % passants

83 %

Risques OWASP couverts
(objectif 75 % dépassé)

3

Environnements définis
TEST opérationnel via Docker

Points forts

- **Un seul outil** (GitHub) pour code + CI/CD + tickets
- **Tests bloquants** — aucun code cassé vers le TEST
- **Sécurité en couches** — natif + correctifs ciblés
- **RGPD dès la conception** — minimisation + droits utilisateurs

Perspectives v1.1.0

- CD automatisé vers la production (approbation manuelle)
- Monitoring Sentry + Uptime Robot
- Export RGPD — droit à la portabilité
- Hachage des tokens API (R3)
- Hébergement HDS pour la mise en production réelle

Merci de votre attention — Questions ?